ToggleMaster em microsserviços na AWS

Relatório de entrega — Tech Challenge Fase 2, POSTECH DevOps

Nome	**Cássio Murilo dos Santos**
RM	375884
Usuário do Discord	cassiomurilo
Repositório (Fase 2)	github.com/cmurilosan/cmurilosan-tech-challenge-fase2-togglemaster
Vídeo da demonstração	youtu.be/ec564st6UB0
Arquivos da entrega	pasta no Google Drive — alternativa de acesso ao vídeo e aos documentos
Repositório (Fase 1)	github.com/cmurilosan/tech-challenge-fase1-togglemaster

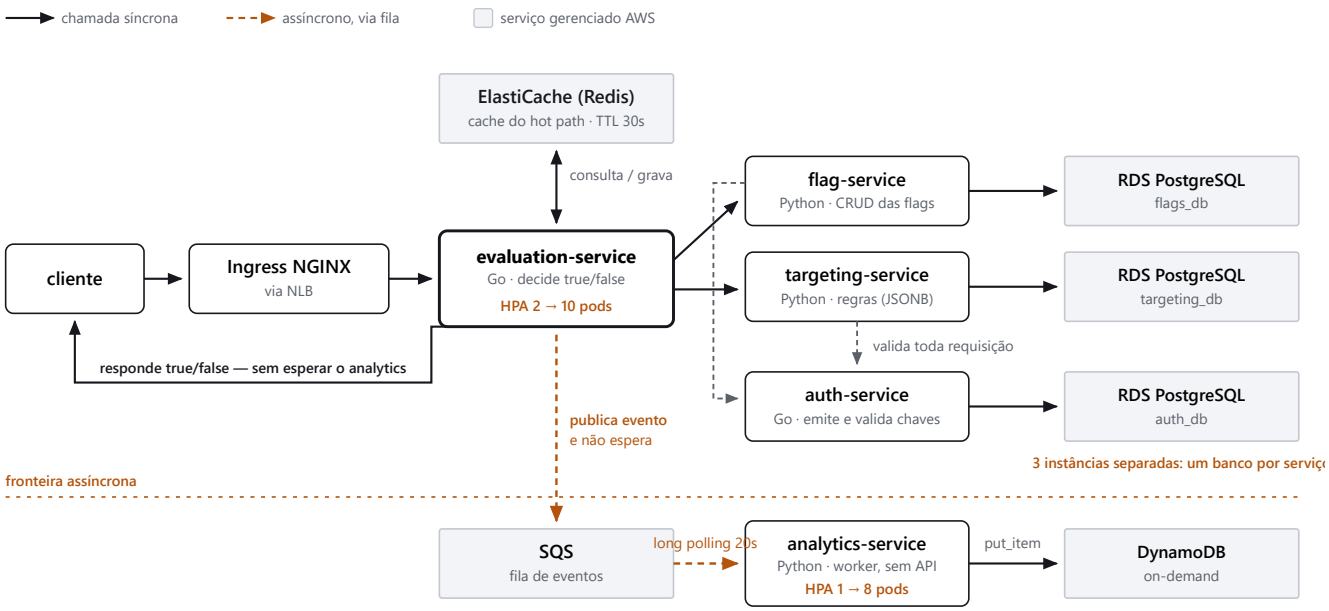
> **Se o vídeo no YouTube não abrir**, a mesma gravação e os documentos desta entrega estão na pasta do Google Drive indicada acima.

O sistema

O ToggleMaster é uma plataforma de *feature flags*: ela responde se um recurso deve estar ligado para um usuário específico. A Fase 1 entregou isso como um monólito; a Fase 2 o decompôs em cinco microsserviços, cada um com seu próprio banco de dados.

SERVIÇO	LINGUAGEM	RESPONSABILIDADE	ESTADO
`auth-service`	Go	Emite e valida chaves de API; toda requisição passa por ele	RDS PostgreSQL próprio
`flag-service`	Python	Cadastro das flags	RDS PostgreSQL próprio
`targeting-service`	Python	Regras de segmentação por usuário	RDS PostgreSQL próprio
`evaluation-service`	Go	Decide se a flag está ligada; cacheia no Redis e publica o evento no SQS	ElastiCache Redis
`analytics-service`	Python	Consome a fila e grava cada avaliação	DynamoDB

Arquitetura



O caminho síncrono vai do cliente ao `evaluation-service`, que responde `true` ou `false` sem esperar o analytics. Abaixo da fronteira tracejada está o caminho assíncrono: o evento vai para o SQS e o `analytics-service` o consome no seu próprio ritmo. É essa separação que permite escalar a escrita de métricas sem afetar o tempo de resposta.

As cinco etapas exigidas

ETAPA	COMO FOI ATENDIDA
1. Containerização	Dockerfile *multi-stage* nos cinco serviços. As imagens em Go ficaram em ~7 MB; as em Python, ~65 MB, porque o interpretador acompanha. O `docker-compose.yml` sobe o sistema inteiro local, com dois PostgreSQL, Redis e DynamoDB Local — mais ElastiMQ, emulador de SQS, já que a AWS não oferece fila local.
2. Infraestrutura AWS	Cluster EKS, 5 repositórios ECR, 3 instâncias RDS PostgreSQL, 1 ElastiCache Redis, 1 tabela DynamoDB *on-demand* e 1 fila SQS Standard. Tudo provisionado manualmente, documentado em `docs/runbook-aws.md`.
3. Add-ons do cluster	Metrics Server (sem ele o HPA não tem o que medir) e Nginx Ingress Controller (`kubernetes/ingress-nginx`), que provisiona um Network Load Balancer como única porta de entrada pública.
4. Manifestos Kubernetes	Namespace, ConfigMap, Secret, cinco Deployments, cinco Services ClusterIP e um Ingress. Todos os contêineres têm `requests` e `limits`, e três sondas: `startupProbe`, `livenessProbe` e `readinessProbe`.
5. Escalabilidade	HPA por CPU no `evaluation-service` (2 a 10 réplicas) e no `analytics-service` (1 a 8), ambos com alvo de 60%.

Decisões técnicas e suas justificativas

Cluster criado pela AWS CLI, não pelo Console

O enunciado descreve o provisionamento "usando o Console" e **proíbe explicitamente o `eksctl`**. A proibição explícita foi respeitada. Optei pela AWS CLI porque ela é igualmente provisionamento manual — cada recurso é um comando meu, não uma automação de terceiros — com a vantagem de ser reproduzível e auditável, o que o clique no Console não é.

HPA por CPU em vez de KEDA

Escalar um trabalhador de fila pelo **tamanho da fila** seria o ideal, e a ferramenta para isso é o KEDA. Ele depende de IRSA, que exige criar uma IAM Role — operação que o AWS Academy não permite. Usei então o caminho que o próprio enunciado indica: escalar por CPU. A lógica é indireta mas sólida: fila cheia significa trabalhador processando, processar consome CPU, a CPU sobe e o HPA cria réplicas. Com a fila vazia o consumidor dorme no *long polling*, a CPU cai e ele reduz. **A CPU funciona como termômetro do tamanho da fila.**

Um banco por serviço

Três instâncias RDS em vez de uma com três *schemas*. É mais caro, e é deliberado: um banco compartilhado recria o acoplamento que a decomposição em microsserviços foi feita para desfazer. Com bancos separados, um serviço não alcança a tabela do outro nem por acidente, e cada um pode evoluir seu esquema sem coordenar com os demais. A escolha do tipo de banco segue o formato do acesso: PostgreSQL onde há relação e consulta, DynamoDB onde o padrão é escrita intensa de eventos independentes.

Sondas: liveness sem banco, readiness com banco

A `livenessProbe` aponta para `/health`, que apenas confirma que o processo responde. A `readinessProbe` aponta para `/ready`, que consulta a dependência real e devolve 503 se ela falhar. A separação é intencional: se o banco estivesse na liveness, uma oscilação do RDS reiniciaria os pods de todos os serviços ao mesmo tempo, transformando um problema pequeno numa queda geral. Como está, um soluço no banco só tira os pods do balanceamento, e eles voltam sozinhos quando o banco volta — sem reinício.

Secret com `stringData`

O enunciado pede que as *secrets* estejam em base64, e no cluster elas estão: `kubectl get secret -o yaml` as devolve codificadas. Escrevi os manifestos com `stringData`, que delega a codificação ao Kubernetes, porque o resultado armazenado é o mesmo e o manifesto fica legível.

> Vale registrar o que isso **não** resolve: base64 é codificação, não criptografia. Qualquer pessoa com acesso ao Secret o decodifica num comando. O que o Secret entrega de verdade é não espalhar a senha pelos manifestos e pelo histórico do Git, e permitir controlar quem o lê. Proteção real seria criptografia no etcd ou o AWS Secrets Manager, fora do escopo desta fase.

Evidências da execução

A infraestrutura foi provisionada e validada de ponta a ponta na AWS. Números observados na execução gravada:

- Cluster EKS com 2 nós `t3.medium`, 9 pods da aplicação em regime.
- *Smoke test* de ponta a ponta pelo Ingress: **5 de 5 passos** — 20 avaliações, 8 retornando `true`, 20 eventos confirmados no DynamoDB.
- HPA sob carga: `evaluation-service` de **2 para 6 réplicas** por CPU direta; `analytics-service` de **1 para 8**, em cascata pelo acúmulo na fila.
- Mais de **134 mil eventos** gravados no DynamoDB durante a demonstração.
- Envio manual de mensagens direto ao SQS, sem passar por nenhum microsserviço, provando que fila e consumidor são independentes da aplicação.

Reprodutibilidade

Dois caminhos, ambos no repositório:

- `docs/runbook-aws.md` — o procedimento manual, comando por comando, com os achados reais de cada execução e uma seção de *troubleshooting*.
- `scripts/provision-aws.sh` — o mesmo procedimento automatizado, do zero ao *smoke test*, em cerca de 15 minutos.

Nenhuma credencial da AWS aparece nos manifestos. Os pods obtêm credencial da `LabRole` pelo serviço de metadados da instância, e os valores que mudam a cada sessão ficam marcados com a palavra `TROQUE` no repositório.

Limitações conhecidas

- **O `analytics-service` crava no teto de 8 réplicas** sob carga, a ~290% de CPU. O `requests.cpu` de `50m` é agressivo demais para o trabalhador: um HPA bem ajustado encontraria equilíbrio abaixo do máximo, como o `evaluation-service` faz. Subir o *request* para ~150m resolveria. Para a demonstração serve, porque evidencia o teto.
- **Sem KEDA e sem *scale-to-zero* por backlog**, pela restrição de IAM do AWS Academy descrita acima.
- **O hop limit do IMDS precisa ser corrigido a cada sessão.** A correção age por instância, não no *launch template*, e o Auto Scaling substitui os nós na virada de sessão do laboratório.